# Relatório de Edge Computing

Projeto CardioIA - Grupo PulseIA
Fase 3: Monitoramento Contínuo - IoT no Pulso do Paciente

## Objetivo e componentes

O módulo Edge do CardioIA simula um vestível destinado ao acompanhamento contínuo de parâmetros relacionados à saúde cardiovascular. O protótipo usa um ESP32, um DHT22 e um botão de pressão. O DHT22 entrega duas medidas por leitura: temperatura em graus Celsius e umidade relativa. O botão representa a detecção de pulsos; cada toque soma uma ocorrência e o programa converte os pulsos de uma janela de dez segundos em batimentos por minuto (BPM). Assim, há dois sensores distintos conforme solicitado: DHT22 e botão de pulso.

As leituras são feitas a cada cinco segundos. Para cada ciclo, o ESP32 confere se os dados do DHT22 são válidos, reúne temperatura, umidade, BPM e o estado de alerta e grava a estrutura `Amostra` na fila local. O alerta é demonstrativo: é ativado quando temperatura é maior ou igual a 38 °C ou quando BPM é maior ou igual a 120. Esses valores apoiam somente a visualização do exercício e não devem ser usados para decisões clínicas.

## Fluxo de funcionamento

- O ESP32 inicializa o Monitor Serial, o DHT22, os botões e o LED.
- O botão `PULSO` registra batimentos simulados. A cada dez segundos, o total é convertido para BPM.
- A cada cinco segundos, o DHT22 é lido e uma nova amostra é adicionada à fila circular local.
- Se a conectividade estiver ativa, o ESP32 se conecta ao Wi-Fi e ao MQTT e tenta publicar as amostras na ordem em que foram coletadas.
- A amostra é excluída apenas após `mqtt.publish()` aceitar localmente a mensagem para transmissão. Uma falha encerra a sincronização e mantém a amostra na fila. Como o protótipo usa QoS 0, esse retorno não comprova que o broker persistiu ou entregou a mensagem.
- O LED fica aceso com MQTT conectado e pisca sem conexão. O Monitor Serial informa leituras, tamanho da fila, modo offline e tentativas de envio.

## Estratégia de resiliência offline

O botão `ONLINE/OFFLINE` alterna a variável booleana `conectividadeAtiva`. No modo offline, Wi-Fi e MQTT são desconectados, mas a coleta continua: cada nova amostra é incluída na fila. Na retomada, o programa inicia a conexão novamente e transmite todas as amostras pendentes, preservando a ordem de coleta.

A fila possui 1.000 posições. Como há uma coleta a cada cinco segundos, ela representa aproximadamente 5.000 segundos, ou 83 minutos, de resiliência sem rede. Esse limite foi escolhido para um protótipo de monitoramento doméstico: evita consumir toda a memória do ESP32 e conserva as informações mais recentes, que são mais úteis para uma visualização imediata. Quando a capacidade é atingida, a leitura mais antiga é substituída e o contador `amostrasDescartadas` é registrado no Serial. Em produto real, essa perda teria de gerar auditoria e política de retenção apropriada.

Cada amostra também recebe um número sequencial. O painel consegue perceber saltos nessa sequência, enquanto o campo `amostras_descartadas` torna visível uma eventual perda por falta de espaço. O identificador do cliente MQTT inclui um sufixo derivado do ESP32, reduzindo colisões entre duas simulações abertas. As tentativas de Wi-Fi são espaçadas em cinco segundos para que uma queda de rede não bloqueie o processamento nem reinicie a conexão continuamente.

## Persistência e limites do simulador

Em uma placa real, a fila pode ser serializada em SPIFFS ou cartão microSD antes da publicação, o que permitiria recuperar pendências após reinicialização. No Wokwi, entretanto, o ambiente do simulador não preserva adequadamente esse armazenamento entre execuções. Por isso, a fila em RAM e o Monitor Serial são a alternativa de resiliência usada nesta entrega: ela permite comprovar coleta offline e sincronização posterior durante uma mesma simulação, sem alegar persistência inexistente.

A documentação do Wokwi descreve o formato de circuito em `diagram.json` e a configuração de projetos usando `wokwi.toml` no VS Code. Referências: formato do diagrama e configuração do projeto.

## Conclusão

O processamento local reduz a dependência de conexão contínua: o dispositivo continua coletando, identifica limites demonstrativos e conserva um histórico limitado para sincronizar depois. A solução mostra o papel de Edge Computing em IoT de saúde, porém ainda é um protótipo acadêmico. A evolução para contexto real exigiria sensores certificados, persistência física, sincronização com confirmação do servidor, criptografia, identificação segura do dispositivo e validação clínica.